UNIVERSIDADE DE TRÁS-OS-MONTES E ALTO DOURO

Trabalho de Integração de Sistemas
Parte 1

Dimensão do grupo: 2 alunos[1]
Ferramentas e tecnologias sugeridas: Visual Studio, C#, ASP.NET, Microsoft SQL Server, Git
Acompanhamento e registo: Git (com commits regulares)[2] Submissão de Trabalhos: Nonio/Inforestudante.

Contexto

A BetStrike é uma empresa fictícia portuguesa do setor das apostas desportivas online fundada em 2019, com sede em Vila Real. A plataforma permite a utilizadores registados colocar apostas em jogos de futebol de diversas competições nacionais e europeias, desde a Primeira Liga até à Liga dos Campeões UEFA.

Ao longo dos últimos anos, a empresa cresceu de forma acelerada, passando de alguns milhares para mais de 200 000 utilizadores ativos mensais. Com esse crescimento surgiram problemas estruturais graves: os resultados dos jogos continuam a ser inseridos manualmente por operadores, o sistema de pagamentos não comunica diretamente com a plataforma de apostas, e a resolução das apostas depois de um jogo terminar demora entre 15 a 30 minutos — um tempo inaceitável no contexto atual, em que os utilizadores esperam feedback imediato.

1 Os grupos devem ser registados em: https://forms.microsoft.com/e/g94Zm1pdqu
2 Obrigatória a inclusão da equipa docente no projeto (jpsousa@utad.pt; hparedes@utad.pt)

Adicionalmente, a empresa não tem qualquer visibilidade em tempo real sobre o volume apostado, a exposição financeira por jogo ou a margem operacional. As decisões de gestão são tomadas com base em relatórios gerados manualmente no final do dia, o que impede uma resposta ágil a situações anómalas.

O departamento de IT da BetStrike recebeu um mandato para modernizar a arquitetura da plataforma, resolvendo os problemas de integração existentes e preparando o sistema para suportar o crescimento previsto para os próximos anos. Este trabalho tem como objetivo implementar essa modernização em três fases progressivas, cobrindo a integração de sistemas, a expansão com novas tecnologias de middleware e, numa componente exploratória, a adoção de containers para padronizar o ambiente de desenvolvimento e deployment.

O objetivo deste trabalho é desenvolver uma solução de integração robusta que permita a comunicação automática entre estes sistemas heterogéneos. A solução deve garantir a transferência eficiente e fiável dos dados da produção, assegurando a integridade dos mesmos e evitando a duplicação de informação. Pretende-se que esta integração seja realizada utilizando tecnologias de middleware, possibilitando a sincronização em tempo (quase) real dos dados de produção para análise e tomada de decisão.

Arquitetura geral

A solução a desenvolver é composta por três sistemas principais que devem comunicar entre si de forma automatizada:

- Aplicação Geradora de Dados: responsável por simular os resultados dos jogos.
- Plataforma de Resultados de Futebol: sistema web com API REST responsável por receber e disponibilizar os dados dos jogos. Os dados são fornecidos por uma aplicação geradora que simula uma liga real e invoca os endpoints desta plataforma para inserir e atualizar resultados.
- Plataforma de Gestão de Apostas: sistema web central com API REST que gere jogos, resultados e apostas dos utilizadores. Consome os dados da Plataforma de Resultados e expõe os seus próprios endpoints para as aplicações cliente.
- Sistema de Pagamentos: sistema independente que gere o saldo dos utilizadores e processa as transações financeiras resultantes das apostas. Comunica com a Plataforma de Apostas através de Triggers e Stored Procedures.

INFORMAÇÃO TRANSCRITA DO PDF DISPONÍVEL NO INFORESTUDANTE

Parte 1 — Integração REST e Base de Dados (8 valores)

Aplicação Geradora de Dados

A geração de dados é realizada por uma aplicação dedicada (consola ou GUI) que simula os resultados da 1ª Liga (liga de futebol portuguesa) e os publica na Plataforma de Resultados invocando os seus Web Services REST. Esta aplicação deverá:

- Gerar o emparelhamento dos 9 jogos da jornada de forma aleatória no arranque, garantindo que nenhuma equipa é emparelhada consigo própria e que cada equipa aparece exactamente uma vez como casa e uma vez como fora ao longo de duas jornadas consecutivas
- Atribuir a cada jogo um identificador único no formato FUT-AAAA-JJNN, onde AAAA é o ano corrente, JJ é o número da jornada com dois dígitos e NN é o número do jogo dentro da jornada (ex: FUT-2025-0103 para o jogo 3 da jornada 1)
- Simular todos os 9 jogos da jornada em paralelo, publicando atualizações de estado e marcador em intervalos 10 segundos (corresponde a 10 minutos reais)
- Cada jogo deve percorrer os estados pela seguinte ordem: Agendado → Em Curso → Finalizado
- No estado Em Curso, a aplicação deve gerar golos de forma aleatória para cada equipa, com uma probabilidade média de 2 a 3 golos por jogo, incrementando o marcador progressivamente até o jogo transitar para Finalizado
- Invocar o endpoint de inserção da Plataforma de Resultados quando um novo jogo é criado, e o endpoint de atualização sempre que o estado ou o marcador mudam

Dados gerados por jogo

- ID do Jogo: código único no formato FUT-AAAA- JJNN
- Data do Jogo
- Hora de Início
- Equipa Casa: nome da equipa anfitriã
- Equipa Fora: nome da equipa visitante
- Golos Casa: número inteiro de golos marcados pela equipa da casa ao longo dos 90 minutos

- Golos Fora: número inteiro de golos marcados pela equipa visitante ao longo dos 90 minutos
- Estado do Jogo: valor inteiro entre 1 e 5, correspondendo a Agendado (1), Em Curso (2), Finalizado (3), Cancelado (4) ou Adiado (5)

Plataforma de Resultados de Futebol

A Plataforma de Resultados é um sistema web que recebe dados da aplicação geradora e os disponibiliza para consumo por outros sistemas. Expõe uma API REST com os seguintes endpoints:

- recebe os dados de um novo jogo e insere-o no sistema; devolve o identificador interno atribuído e rejeita o pedido se o Codigo_Jogo já existir
- atualiza o estado e/ou o marcador de um jogo existente identificado pelo seu código; devolve erro se o jogo não existir ou se a transição de estado for inválida
- devolve a lista de todos os jogos, com suporte a filtros por data, estado passados como parâmetros de query string
- devolve os dados completos de um jogo específico, incluindo o resultado atual
- remove um jogo do sistema; só é permitido para jogos no estado Agendado

Plataforma de Gestão de Apostas

A Plataforma de Gestão de Apostas suporta-se numa base de dados denominada "Apostas" que persiste toda a informação relativa a jogos, resultados e apostas dos utilizadores. Esta base de dados é composta por três entidades principais: Jogo, Resultado e Aposta.

Entidade Jogo

Cada registo representa um jogo de futebol sincronizado a partir da Plataforma de Resultados. Deve guardar o código único do jogo (FUT-AAAA-JJNN), a data e hora de início, os nomes das duas equipas (casa e fora), o tipo de competição e o estado atual (1 a 5). A chave primária é um identificador

inteiro autoincrementado internamente. O código do jogo deve ter restrição de unicidade para evitar duplicados.

Entidade Resultado

Cada registo associa-se a um único jogo (relação um-para-um) e armazena o resultado final: o total de golos marcados pela equipa da casa e o total de golos marcados pela equipa visitante, ambos ao longo dos 90 minutos de jogo. Inclui ainda a data e hora da última atualização do resultado. A chave estrangeira para a entidade Jogo deve ter restrição de integridade referencial com eliminação em cascata.

Entidade Aposta

Cada registo representa uma aposta individual realizada por um utilizador sobre um jogo específico. Deve guardar a referência ao jogo apostado, o identificador do utilizador, o tipo de aposta (1, X ou 2), o montante apostado em euros, a odd em vigor no momento do registo, o estado da aposta (1=Pendente, 2=Ganha, 3=Perdida, 4=Anulada) e a data e hora de realização. A odd deve ser guardada no momento do registo porque pode variar ao longo do tempo e é este valor que determina o prémio final a pagar.

Tipos de Aposta

Estão disponíveis três tipos de aposta correspondendo ao mercado clássico 1X2 do futebol, resolvidos sempre com base no resultado final dos 90 minutos:

- 1- Vitória da equipa da casa.
- X- Empate.
- 2- Vitória da equipa visitante.

API REST da Plataforma de Gestão de Apostas

A Plataforma de Gestão de Apostas expõe uma API REST que suporta as seguintes operações:

Gestão de Jogos

- insere um novo jogo recebido da Plataforma de Resultados; valida o formato do código e a unicidade antes de persistir

- atualiza estado e resultado de um jogo; quando o estado transita para Finalizado (3), despoleta automaticamente a resolução das apostas pendentes associadas
- lista jogos com filtros opcionais por data, estado e tipo de competição
- devolve dados completos de um jogo, incluindo número de apostas pendentes e volume total apostado
- remove um jogo; só permitido se não existirem apostas associadas e o estado for Agendado

Gestão de Apostas

- regista uma nova aposta; valida o saldo do utilizador, o estado do jogo e o tipo de aposta antes de persistir; debita imediatamente o saldo
- lista apostas com filtros por utilizador, jogo, estado e intervalo de datas
- devolve os detalhes de uma aposta específica, incluindo o prémio potencial calculado
- cancela uma aposta pendente; só é permitido enquanto o jogo ainda estiver no estado Agendado; devolve o valor apostado ao saldo do utilizador

Gestão de Utilizadores

- Criar Utilizador: Regista um novo utilizador no sistema. Esta operação é crítica pois deve garantir a atomicidade entre sistemas: ao criar o utilizador na base "Apostas", deve invocar automaticamente a lógica necessária para criar o registo de Saldo_Utilizador na base "Pagamentos" com um saldo inicial de 50.00€ (promoção de plataforma).

Gestão de Resultados

- insere o resultado final de um jogo; só é permitido uma vez por jogo e apenas quando o estado for Finalizado
- devolve o resultado de um jogo específico

Estatísticas

- devolve estatísticas de um jogo: total apostado, número de apostas por tipo, número de apostas por estado e margem da plataforma
- devolve estatísticas agregadas para um tipo de competição: média de golos por jogo, volume total apostado e taxa de vitória de cada resultado (1, X, 2)

INFORMAÇÃO TRANSCRITA DO PDF DISPONÍVEL NO INFORESTUDANTE

Acesso à Base de Dados via Stored Procedures

Todo o acesso à base de dados, incluindo inserções, atualizações e consultas, é feito exclusivamente através de Stored Procedures. Esta abordagem serve três propósitos: centralizar a lógica de validação na camada de dados, proteger o esquema contra acessos diretos e facilitar a auditoria de operações. A API não deve executar SQL inline — todas as operações devem invocar a Stored Procedure correspondente.

Devem ser criadas Stored Procedures para as seguintes operações: inserir jogo, atualizar estado e resultado de jogo, inserir aposta, cancelar aposta, resolver apostas de um jogo finalizado, inserir resultado, consultar jogos com filtros, consultar apostas por utilizador e consultar estatísticas por jogo e por competição.

Regras de Validação

As seguintes regras de validação devem ser implementadas nas Stored Procedures e reforçadas na camada de API:

- Não podem existir dois jogos com o mesmo código FUT-AAAA-JJNN; tentativas de inserção duplicada devem ser rejeitadas com erro descritivo • Um resultado só pode ser inserido para um jogo no estado Finalizado (3); um jogo só pode ter um resultado. Tentativas de inserir um segundo resultado devem ser rejeitadas
- Se um jogo transitar para Finalizado sem que exista resultado associado, o sistema deve inserir automaticamente um resultado com Golos_Casa=0 e Golos_Fora=0, sinalizando o resultado como desconhecido
- Não é permitido registar apostas em jogos com estado Finalizado (3), Cancelado (4) ou Adiado (5)
- O tipo de aposta deve ser obrigatoriamente 1, X ou 2, qualquer outro valor deve ser rejeitado
- O valor apostado deve ser um número positivo maior que zero
- A odd registada no momento da aposta deve ser maior que 1.0
- O saldo do utilizador deve ser suficiente para cobrir o valor apostado, se não for, a aposta deve ser rejeitada e o saldo não é alterado

INFORMAÇÃO TRANSCRITA DO PDF DISPONÍVEL NO INFORESTUDANTE

- As transições de estado de um jogo devem respeitar a sequência válida: um jogo Finalizado não pode voltar a Em Curso; um jogo Cancelado ou Adiado não pode mudar de estado

Sistema de Pagamentos

A base de dados "Pagamentos" é um sistema independente que gere todas as movimentações financeiras dos utilizadores da plataforma. Está separada da base de dados "Apostas" e comunica com ela exclusivamente através de Triggers e de chamadas explícitas a Stored Procedures.

Entidade Transação

Cada registo representa uma movimentação financeira e deve guardar: a referência à aposta que originou a transação (quando aplicável), o identificador do utilizador, o tipo de transação, o valor em euros, a data e hora e o estado do processamento (Pendente, Processada, Falhada ou Reembolsada). Existem cinco tipos de transação: AP (Débito de Aposta), PG (Pagamento de Prémio), RE (Reembolso), DE (Depósito) e LV (Levantamento).

Entidade Saldo_Utilizador

Mantém o saldo disponível de cada utilizador. Deve guardar o identificador do utilizador (único), o saldo atual em euros e a data e hora da última atualização. O saldo nunca pode ser negativo — qualquer operação que resulte em saldo negativo deve ser rejeitada antes de ser executada.

Gestão de Saldo e Testes

Para permitir o teste do fluxo completo de apostas, o Sistema de Pagamentos deve disponibilizar uma forma de Depósito de Dinheiro Fictício. Opcionalmente, pode ser exposto um endpoint na API para facilitar esta operação durante os testes de integração.

Integração com o Sistema de Pagamentos via Trigger

A comunicação entre a base de dados "Apostas" e a base de dados

"Pagamentos" é feita através de um Trigger que dispara automaticamente sempre que o estado de uma aposta é alterado. O Trigger deve executar a seguinte lógica consoante o novo estado:

- Aposta Ganha (2): criar uma transação do tipo PG com o valor calculado como Valor_Apostado × Odd_Momento, e adicionar esse valor ao saldo atual do utilizador
- Aposta Anulada (4): criar uma transação do tipo RE com o valor igual ao Valor_Apostado, e adicionar esse valor ao saldo atual do utilizador
- Aposta Perdida (3): não criar qualquer transação; o valor já tinha sido debitado no momento em que a aposta foi registada

O débito do valor apostado (transação do tipo AP) é criado no momento em que a aposta é registada, não quando é resolvida. O Trigger é apenas responsável pelo crédito posterior em caso de ganho ou anulação.

Resolução Automática de Apostas

Quando um jogo transita para o estado Finalizado (3), uma Stored Procedure deve ser invocada automaticamente para resolver todas as apostas pendentes associadas a esse jogo. A resolução é feita com base no resultado final dos 90 minutos, comparando Golos_Casa com Golos_Fora:

- Apostas do tipo 1X são marcadas como Ganhas (2) se Golos_Casa for superior a Golos_Fora, ou como Perdidas (3) em caso contrário
- Apostas do tipo XX são marcadas como Ganhas (2) se Golos_Casa for igual a Golos_Fora, ou como Perdidas (3) em caso contrário
- Apostas do tipo X2 são marcadas como Ganhas (2) se Golos_Fora for superior a Golos_Casa, ou como Perdidas (3) em caso contrário

Se o jogo transitar para Cancelado (4) ou Adiado (5), todas as apostas pendentes associadas devem ser marcadas como Anuladas (4), despoletando o Trigger de reembolso para cada uma.

INFORMAÇÃO TRANSCRITA DO PDF DISPONÍVEL NO INFORESTUDANTE

Parte Exploratória — Containerização da Infraestrutura (3 valores)

Esta parte tem carácter exploratório e destina-se a alunos que pretendam aprofundar os seus conhecimentos de forma autónoma. A sua realização é opcional mas valorizada.

Contexto e Motivação

À medida que o número de serviços de uma aplicação cresce, a gestão do ambiente de desenvolvimento torna-se cada vez mais complexa. Instalar e configurar manualmente o SQL Server, o RabbitMQ e as APIs em cada máquina é um processo demorado, sujeito a erros e que produz resultados diferentes consoante o sistema operativo ou as versões instaladas. A tecnologia de containers resolve este problema encapsulando cada serviço e todas as suas dependências numa unidade isolada e portátil que se comporta de forma idêntica em qualquer máquina.

O objetivo desta parte é containerizar todos os serviços de backend do sistema desenvolvido nas Partes 1 e 2, utilizando Docker e Docker Compose, de forma a que a infraestrutura completa possa ser levantada com um único comando.